A Real-Map Rideshare Dispatch Benchmark with Strong Heuristic Baselines

Ahmad Alashoury **Koroush Emari** **Abdelrahman El Banna** **Shikar Srivastava**

Abstract

We study pooled rideshare dispatch on a real OpenStreetMap extract of downtown London, Ontario. The benchmark is intentionally more realistic than a toy pickup-and-dropoff problem: one vehicle with two seats must serve four riders while handling traffic phases, deadlines, and pooling quality. Our contribution is primarily a *benchmark and formulation* rather than a new RL update rule. We introduce a phase-aware, hub-level dispatch MDP that separates high-level stop selection from low-level path following: the agent chooses the next pickup or dropoff stop, and the environment executes the shortest path on the real street graph. We compare Q-learning, SARSA, DQN, and PPO against two simple non-RL baselines, nearest-stop and earliest-deadline-first. Over three random seeds, DQN is the strongest learned policy at 58.3% on-time riders, 1.67 late dropoffs, and 43.4 steps per episode, but a nearest-stop heuristic is substantially stronger overall at 83.8% on-time riders and 35.9 steps. A focused DQN ablation also shows that curriculum learning and action masking are not the main source of performance on this benchmark. The main conclusion is therefore not that RL wins, but that the benchmark is useful precisely because it reveals when structured heuristics remain hard to beat.

1 Introduction

Urban rideshare systems must make decisions under uncertainty: which rider should be picked up first, when should the platform pool riders together, and how should congestion affect the route order? The core challenge is not only path finding. It is a sequential service problem in which local decisions can make later riders late, reduce pooling efficiency, or force expensive detours.

This project studies that decision problem as a reinforcement learning (RL) task. The setting is a single Uber-style vehicle operating on a real downtown London, Ontario street graph derived from OpenStreetMap. The vehicle has two seats and must serve four riders. Each rider has a pickup hub, a dropoff hub, and a delivery-style deadline. The environment also includes traffic phases representing commute, event, and nightlife congestion.

The problem is difficult for existing techniques for three reasons. First, exact dynamic assignment methods can optimize a given batch of trips, but they typically require hand-crafted objectives, known demand, or repeated re-solving and do not naturally learn reusable dispatch policies [Alonso-Mora et al., 2017]. Second, tabular RL methods suffer from rapid state growth once multiple riders, deadlines, and traffic contexts are included [Watkins and Dayan, 1992, Rummery and Niranjan, 1994]. Third, low-level control methods spend most of their capacity on driving mechanics when the research question is really about dispatch order.

Our intuition is that the rideshare problem becomes both realistic and learnable if we factor it into two layers. The environment should handle shortest-path routing on a real map, while the decision policy should learn the *next stop selection problem*. We implement this idea as a phase-aware, hub-level Markov decision process (MDP), then compare Q-learning, SARSA, DQN, and PPO against

Preprint.

simple handcrafted heuristics. This framing makes the paper stronger: the contribution is a map-grounded benchmark that lets us ask not only which RL method is best, but also whether RL is necessary at all in this compact dispatch setting.

2 Techniques To Tackle the Problem

2.1 Previous Work

Classical RL baselines remain useful reference points for dispatch problems. Q-learning learns action values by bootstrapping toward the greedy next action [Watkins and Dayan, 1992], while SARSA performs an on-policy update using the next action actually taken [Rummery and Niranjan, 1994]. Deep Q-Networks (DQN) replace tabular value storage with a neural approximation and replay buffer, enabling generalization across related states [Mnih et al., 2015]. PPO provides a stable policy-gradient baseline through clipped policy updates [Schulman et al., 2017]. In practice, however, rideshare systems are also often approached with simple rule-based dispatch heuristics, so any learned policy should ideally be judged against those baselines too.

Transportation and fleet-management work has shown that realistic dispatching and pooling create large, stochastic decision spaces. Alonso-Mora et al. formulate dynamic trip–vehicle assignment for high-capacity ride-sharing and show the value of explicit assignment optimization [Alonso-Mora et al., 2017]. Oda and Joe-Wong propose MOVI, a model-free DQN-based fleet management framework, highlighting that learned policies can outperform handcrafted dispatch baselines at scale [Oda and Joe-Wong, 2018]. Lin et al. extend this direction with contextual multi-agent deep RL for large-scale fleet management [Lin et al., 2018]. These papers motivate our project, but they target large-fleet or centralized rebalancing settings rather than a compact, presentation-friendly benchmark where individual dispatch choices remain interpretable.

2.2 Our Technique

The main technique we developed is not a brand-new RL algorithm. Instead, it is a **dispatch benchmark and formulation** that makes a realistic rideshare problem tractable for standard RL baselines. It has four parts.

(1) Real-map, hub-level abstraction. We start from a real OpenStreetMap extract of downtown London. We rasterize the road network into a 28×28 grid, keep the largest connected component, and anchor named hubs such as London Station, Covent Garden Market, Canada Life Place, London City Hall, Victoria Park, Museum London, Grand Theatre, Central Library, Fanshawe Downtown, and Armouries Hotel. The resulting benchmark contains 335 road cells, 10 hubs, and 3 start depots.

(2) Dispatch instead of driving. The agent does not choose north, south, east, or west. It chooses one of eight high-level actions: pickup rider $1 \ldots 4$ or dropoff rider $1 \ldots 4$. Once the agent chooses a stop, the environment executes the shortest path over the real road graph. This design focuses learning on stop ordering and service quality instead of low-level locomotion.

(3) Phase-aware congestion and clustered demand. Each episode activates one traffic phase: *Commute Rush*, *Event Surge*, or *Nightlife Rush*. Phase-specific penalty zones are layered on top of the base hotspot roads. Rider demand is also phase-conditioned through three templates: *Commute Wave*, *Event Exit*, and *Culture Loop*. This creates structured scenarios in which the same dispatch policy should adapt differently under different traffic and rider mixes.

(4) Optional training scaffolding. We also study two common engineering supports: legal-action masking and easy-to-hard curriculum training. Illegal actions can be masked so that riders cannot be picked up when the car is full or dropped off before pickup. Curriculum learning begins with two-rider, low-phase episodes and then increases to the full benchmark. These components were originally intended to stabilize learning; in Section 4 we test whether they actually matter.

2.3 Benchmark Construction Pipeline

The benchmark is generated in four explicit stages. First, we parse a real OpenStreetMap export of downtown London and keep only drivable street segments. Second, we project those coordinates onto a 28×28 lattice, rasterize road segments into grid cells, and retain the largest connected component to avoid disconnected fragments. Third, we snap named downtown landmarks to their nearest usable road cells and promote them to hubs or depots. Finally, we overlay benchmark-specific logic: traffic hotspots, phase-dependent congestion regions, clustered rider-demand templates, and deadline heuristics.

This pipeline matters because it explains where the realism comes from. The *geometry* is real: hubs are placed on an actual downtown road skeleton. The *decision problem* is custom: traffic phases, pooling bonuses, and demand templates are engineered to create recurring but nontrivial dispatch patterns. In other words, we are not claiming to simulate Uber exactly. We are building a realistic, reproducible RL benchmark whose structure can be inspected and justified.

3 Problem Formulation

3.1 State

For tabular methods, the symbolic state is

$$s = (\text{position}, \text{phase}, \text{carrying count}, \text{request signature}),$$

where each rider contributes a tuple containing pickup hub, dropoff hub, status, deadline bucket, and wait bucket. Rider status is one of $\{\text{waiting}, \text{onboard}, \text{delivered}\}$.

For neural methods, we use a 187-dimensional feature vector. It includes one-hot current location over 11 mobility nodes, one-hot carrying count, one-hot traffic phase, per-action legality and normalized target-distance features, per-rider structured features, and global progress indicators such as delivered fraction, late-dropoff fraction, peak occupancy, and step fraction.

The dual representation is an engineering compromise. The compact symbolic state allows Q-learning and SARSA to remain feasible on a laptop, while the dense 187-dimensional observation gives DQN and PPO enough structure to exploit similarities across rider configurations. This is also part of the comparison logic: if the benchmark is truly structured, the neural methods should be able to use these richer features to generalize better than the tabular baselines.

3.2 Actions

There are eight actions:

$$\{\text{pickup}_1, \ldots, \text{pickup}_4, \text{dropoff}_1, \ldots, \text{dropoff}_4\}.$$

Action masking enforces simple domain rules. A pickup is legal only if the rider is still waiting and capacity is available. A dropoff is legal only if the rider is onboard.

3.3 Transition Dynamics and Traffic

After the agent selects a legal action, the environment computes the shortest path from the current mobility node to the action's target hub and executes that path cell by cell. Time advances on every traversed road cell. During that travel, each unfinished rider loses one unit of deadline budget; riders not yet picked up accumulate waiting time, while riders already onboard accumulate ride time. Once the vehicle reaches the target hub, the corresponding pickup or dropoff transition is applied.

Traffic is modeled as an additive phase-dependent traversal penalty rather than as a separate microscopic vehicle simulation. The map contains base hotspot roads inferred from frequent street usage during rasterization, and each episode activates one of three phase maps: *Commute Rush*, *Event Surge*, or *Nightlife Rush*. These phases raise the cost of traveling near specific landmark clusters such as London Station, Canada Life Place, Grand Theatre, and Covent Garden Market. This choice keeps the state compact while still making dispatch order phase-sensitive.

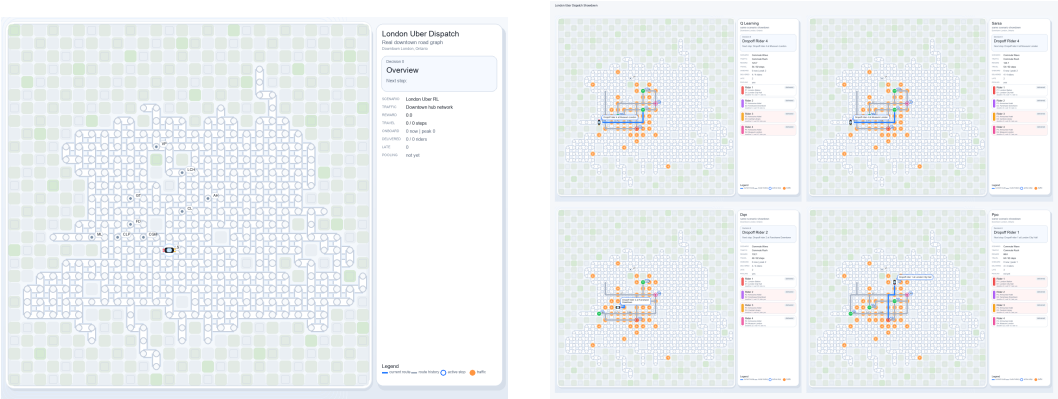

Figure 1: Left: the processed London benchmark map. Right: a same-scenario qualitative comparison of the four learned policies.

3.4 Deadlines, Waiting Time, and Scenario Generation

Each rider is assigned a deadline budget at the start of the episode. The budget is computed from three pieces: distance from the depot to the rider's pickup, distance from pickup to dropoff, and an additional slack term controlled by the curriculum stage. This means deadlines are not arbitrary; they are tied to the actual geometry of the scenario. A rider is marked late only if the vehicle completes the dropoff after that deadline budget reaches zero.

Demand is generated from clustered templates rather than uniform random sampling. For example, an event-phase episode samples riders near event-related hubs with a bias toward realistic pickup/dropoff pairings. This choice increases the chance of meaningful pooling and makes the same dispatch policy behave differently under commute, event, and nightlife conditions. Without such structure, the environment would mostly test generic path-finding rather than rideshare reasoning.

3.5 Reward

The reward is shaped around service quality. For each road cell traversed, the environment applies:

- travel cost: -0.18,
- unfinished-workload penalty: $-0.04\times$ remaining undelivered riders,
- traffic penalty for the active phase,
- pooling bonus: $+0.12 \times \max(0, \text{occupancy} - 1)$.

Successful pickups receive $+10$, with an extra $+4$ if the vehicle reaches occupancy two. On-time dropoffs receive $+28$, while late dropoffs receive $+18$. Invalid actions are penalized by -8, and timing out at the episode limit adds -12. Finishing all riders gives $+22$, with bonuses for zero late riders $(+10)$ and successful pooling $(+8)$.

The reward was designed to encode three priorities at once. First, service should complete successfully, hence the large dropoff and terminal rewards. Second, service should be timely, hence the difference between on-time and late dropoffs plus the ongoing deadline pressure. Third, the policy should behave like a pooled rideshare driver rather than a sequence of isolated taxi trips, hence the occupancy-based shaping term and final pooling bonus. A pure shortest-path objective would miss all three of these priorities.

Table 1: Three-seed evaluation results on the London dispatch benchmark. All methods achieved 100% completion, so we report service-quality metrics. Values are mean ± standard deviation across seeds for on-time riders.

Method	Type	On-time (%)	Late dropoffs	Steps
Q-learning	RL	53.3 ± 1.2	1.87	45.55
SARSA	RL	52.1 ± 2.1	1.92	45.78
DQN	RL	58.3 ± 5.1	1.67	43.43
PPO	RL	56.7 ± 1.6	1.73	45.43
Nearest-stop	heuristic	**83.8 ± 6.7**	**0.65**	**35.95**
Earliest-deadline	heuristic	67.9 ± 1.6	1.28	49.35

4 Evaluation

4.1 Experimental Setup

We compare Q-learning, SARSA, DQN, and PPO, and we additionally evaluate two hand-crafted baselines: nearest-stop and earliest-deadline-first. The report uses 3 random seeds, 900 training episodes per seed for the learned methods, and 20 greedy evaluation episodes per seed. We report mean reward, full-success rate, on-time rider rate, mean late dropoffs, pooling rate, peak occupancy, and mean evaluation steps.

The algorithm settings are intentionally comparable rather than heavily tuned. Q-learning uses learning rate 0.18 and discount 0.99; SARSA uses 0.14 and 0.985; DQN uses a 128-unit hidden layer, learning rate 0.0011, replay size 16,000, batch size 96, and target synchronization every 180 updates; PPO uses a 128-unit policy/value network, policy learning rate 0.001, value learning rate 0.002, 8 rollout episodes per update, and 8 optimization epochs.

We evaluate policies greedily so that the test-time metrics reflect the learned policy rather than exploration noise. The metrics are chosen to separate *completion* from *quality*. Full success rate measures whether all four riders are eventually served. On-time rider rate measures what fraction of served riders beat their deadlines. Late-dropoff count is the complementary penalty-focused metric. Pooling rate records whether the vehicle ever reached occupancy two, peak occupancy summarizes how aggressively the learned policy pooled riders, and step count measures route efficiency. This metric set matters because a benchmark in which all algorithms achieve 100% completion can still reveal large practical quality differences. The role of the hand-crafted baselines is equally important: if a simple geometric rule already solves the benchmark extremely well, then the correct conclusion is not "RL wins," but rather that the environment still favors local dispatch heuristics.

4.2 Quantitative Results

Table 1 summarizes the main results. All six methods reach 100% task completion on this benchmark, so completion itself is not the meaningful separator. The interesting differences appear in service quality.

4.3 Analysis

The central empirical result changes the paper's interpretation. DQN is the *best learned policy*: it has the highest on-time rider rate among the RL methods, the fewest late dropoffs, and the lowest evaluation step count. PPO is the closest learned competitor, while Q-learning and SARSA lag on timeliness. However, the strongest overall method is not RL at all. The nearest-stop heuristic outperforms every learned policy by a large margin.

That finding is informative rather than disappointing. It suggests that the current benchmark still rewards local geometric structure very strongly. Because the environment executes shortest-path travel automatically and demand is spatially clustered, a heuristic that repeatedly chooses the nearest legal stop already captures much of the useful structure. In contrast, earliest-deadline-first performs worse than nearest-stop and does not pool riders, which shows that not all simple heuristics are

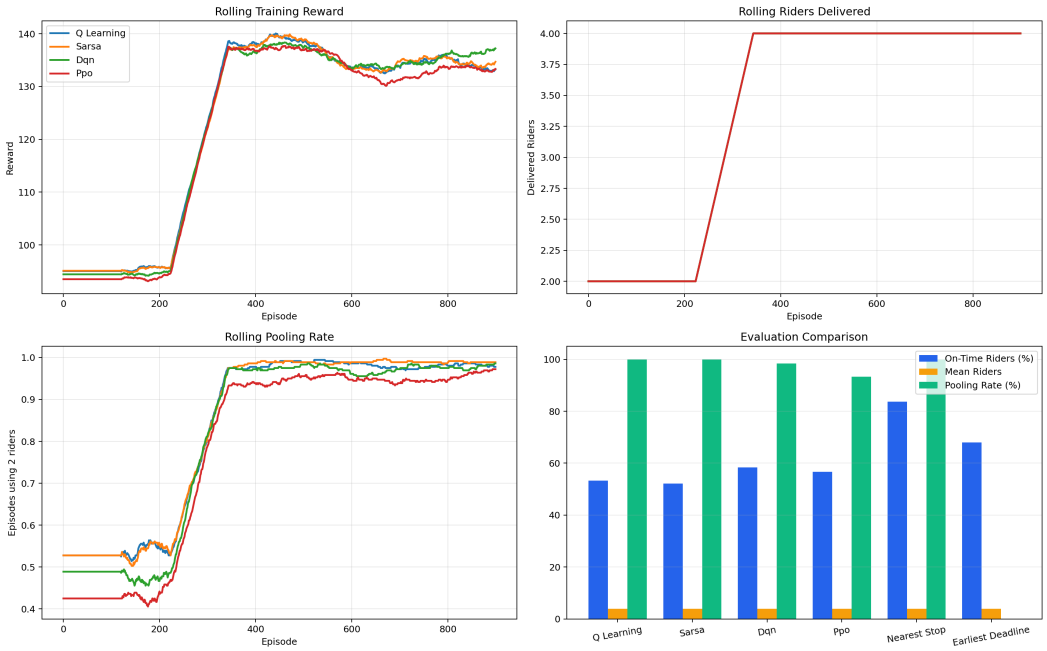

Figure 2: Rolling training curves and aggregated evaluation results across the four algorithms.

equally good. The benchmark therefore does reveal meaningful tradeoffs, but it also shows that RL is not yet necessary to achieve strong performance.

The revised conclusion is more modest and more credible: our benchmark is useful because it distinguishes between learned methods, but also because it reveals when a compact rideshare problem is still dominated by simple local rules. If we had reported only RL-vs-RL comparisons, we would have missed that much stronger conclusion.

4.4 Ablation Study

We also ran a focused DQN ablation to test two engineering choices that might otherwise be overcredited: curriculum learning and action masking. In a separate three-seed run, the full DQN model reached 66.3% on-time riders with 1.35 late dropoffs and 41.5 mean steps. Removing action masking changed little: 67.9% on-time riders, 1.28 late dropoffs, and 41.6 steps. Surprisingly, removing curriculum improved the result to 75.0% on-time riders, 1.00 late dropoffs, and 39.0 steps.

This is a negative result, but an important one. It suggests that on the current benchmark, neither curriculum learning nor masking is the main reason DQN works. Instead, the dominant factor appears to be the map geometry and clustered demand itself. That makes the benchmark more interpretable, but it also means we should avoid claiming that our training scaffolding is a key algorithmic contribution.

4.5 Qualitative Behavior

The qualitative comparison in Figure 1 still helps explain the learned-method ranking. DQN typically commits to more direct stop orderings once a phase-aware pattern has been recognized. PPO often finds similar routes but is somewhat less consistent about pooling two riders early in the episode. The tabular baselines can still solve the route set, but they more often choose service orders that create one or two extra detours, which then show up as higher step counts and more late riders.

Since the environment handles shortest-path travel automatically, any route inefficiency in the visualization comes from the *choice of next stop*, not from low-level navigation error. That is exactly the behavior we wanted to compare. At the same time, the strong nearest-stop heuristic implies that

many good choices are still recoverable from simple local geometry, which helps explain why the benchmark does not yet strongly favor learned policies over handcrafted ones.

4.6 Threats to Validity

Our conclusions should be interpreted with a few limitations in mind. First, the benchmark uses a single vehicle and fixed rider batch per episode, so it is closer to compact pooled dispatch than to full marketplace simulation. Second, traffic is represented as additive road-cell penalties rather than dynamic travel-time uncertainty. Third, the present evaluation uses three seeds and moderate training budgets, which is enough to expose stable trends but not enough to claim fully tuned state-of-the-art performance.

5 Conclusion

We developed a phase-aware pooled rideshare dispatch benchmark on a real OpenStreetMap extract of downtown London, Ontario. The key idea was to let the environment handle shortest-path routing while learned or handcrafted policies solve the next-stop dispatch problem. This makes the problem realistic enough to be interesting, but still compact enough to inspect.

Our strongest conclusion is not that RL solves the problem best. Instead, the benchmark reveals two things at once: DQN is the best *learned* policy among the compared RL methods, but a simple nearest-stop heuristic is stronger overall. The ablation study further shows that curriculum learning and action masking are not the main source of DQN's performance on this environment. Together, these findings make the paper more useful: the benchmark exposes when learned dispatch helps, when heuristics are still hard to beat, and which engineering choices matter less than expected.

Several limitations remain. The benchmark is single-vehicle, the traffic phase is fixed within an episode, and deadlines are deterministic. Future research should therefore make the environment less favorable to local geometric rules by introducing online rider arrivals, within-episode traffic changes, multi-vehicle coordination, asymmetric rider priorities, and larger batch sizes. Those extensions would create a stronger test of whether learned dispatch truly outperforms well-designed heuristics.

References

Javier Alonso-Mora, Samitha Samaranayake, Alex Wallar, Emilio Frazzoli, and Daniela Rus. On-demand high-capacity ride-sharing via dynamic trip-vehicle assignment. *Proceedings of the National Academy of Sciences*, 114(3):462–467, 2017.

Kaixiang Lin, Renyu Zhao, Zhou Xu, and Jiayu Zhou. Efficient large-scale fleet management via multi-agent deep reinforcement learning. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 1774–1783, 2018.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

Takuma Oda and Carlee Joe-Wong. MOVI: A model-free approach to dynamic fleet management. *arXiv preprint arXiv:1804.04758*, 2018.

Gavin A. Rummery and Mahesan Niranjan. On-line Q-learning using connectionist systems. Technical Report CUED/F-INFENG/TR 166, University of Cambridge, Department of Engineering, 1994.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992.